

Gaming Support Intent Classification with LoRA

Parameter-Efficient Fine-Tuning of Qwen2.5-0.5B-Instruct for Gaming Peripheral Support Intent Classification

Technical Report | Lightweight internship-preparation project | Google Colab, Tesla T4

Objective. Adapt a small open-source instruction-tuned LLM to classify gaming-peripheral support queries into six predefined intents, then compare the base model, LoRA fine-tuning, and LoRA with 4-bit quantized inference.

41.67%	91.67%	+50.00 pp	0.1093%
Base accuracy	LoRA accuracy	LoRA accuracy gain	Trainable parameter ratio

1. Task and Dataset

The task is generative six-class intent classification. For each user query, the model must return exactly one label: device_not_detected, wireless_connectivity, firmware_update, rgb_lighting, audio_issue, or software_configuration.

The dataset contains **600 synthetic, balanced examples** (100 per intent), built from 90 manually curated seed examples and 510 examples produced with controlled template augmentation. Quality checks covered missing values, invalid labels, exact duplicates, class balance, cross-split overlap, and TF-IDF/cosine-similarity inspection for near-duplicates.

Split	Samples	Per class	Purpose
Train	480	80	LoRA supervised fine-tuning
Validation	60	10	Training validation and sanity checks
Test	60	10	Frozen final evaluation

2. Method

Base model

Qwen2.5-0.5B-Instruct was evaluated zero-shot with a fixed system prompt and greedy generation. Predictions were scored by exact string match after lowercase/whitespace normalization; no keyword remapping was used.

LoRA fine-tuning

PEFT LoRA adapters were applied to q_proj and v_proj with **r=8, alpha=16**, and **dropout=0.05**. Only **540,672 parameters** were trainable out of approximately 494.6M total parameters.

Training setup

2 epochs; learning rate 1e-4; effective batch size 16; maximum sequence length 128; FP16 training; completion-only SFT loss. Validation loss decreased from **0.0661** after epoch 1 to **0.0444** after epoch 2, with no NaN training loss.

4-bit inference

The trained adapter was attached to a 4-bit NF4 quantized base model using bitsandbytes. This was **inference-only quantization**, not QLoRA retraining.

3. Results

Model	Accuracy	Macro F1	Valid output	Avg latency	Peak GPU memory
Base Qwen2.5-0.5B (FP16)	41.67%	0.3204	98.33%	362.86 ms	2.800 GB
LoRA fine-tuned (FP16)	91.67%	0.9233	98.33%	295.85 ms	2.802 GB
LoRA + 4-bit	98.33%	0.9912	98.33%	386.98 ms	2.324 GB

Fine-tuning effect

LoRA increased exact-match accuracy by **50.00 percentage points** and Macro F1 by **0.6030**. The base model completely missed `device_not_detected` on the test set (recall 0.00), while LoRA raised recall to 0.80, indicating that task-specific label boundaries were learned.

Memory/latency trade-off

FP16 Base and FP16 LoRA used almost identical peak inference memory (2.800 vs. 2.802 GB), as LoRA still requires the full base model at inference time. The 4-bit variant reduced peak memory by **17.06%** relative to FP16 LoRA, but was slower in the measured end-to-end latency benchmark.

4-bit accuracy result

The 4-bit model scored 98.33%, higher than FP16 LoRA on this 60-example test set. It corrected five FP16 errors while introducing one new error. This should be interpreted cautiously: quantization perturbs token probabilities, and a small test set makes a few changed predictions appear as a large percentage-point shift.

Benchmark definition

Latency was measured over a fixed 30-query subset with batch size 1, including prompt construction, tokenization, generation, and decoding but excluding model loading. CUDA synchronization was used around timed inference. Peak memory used PyTorch CUDA allocated-memory statistics after warm-up.

4. Error Analysis

The FP16 LoRA model made 5 errors. Three recurring patterns were observed:

- **Device detection vs. software configuration:** two `device_not_detected` queries were predicted as `software_configuration`, especially when configuration software was mentioned.
- **Surface-cue confusion:** a wireless headset with one-sided audio was mapped to `wireless_connectivity`; a 2.4 GHz pairing failure was mapped to `device_not_detected`.
- **Out-of-schema generation:** the generative classifier produced semantically plausible but invalid labels such as `voice_issue` and `keyboard_colorization`.

5. Limitations and Conclusion

The benchmark is intentionally small and controlled. The data is synthetic and template-augmented, the test set contains only 60 examples, and latency/memory measurements were collected on a single Google Colab Tesla T4 runtime. Remaining near-duplicate phrasing may also make the task easier than real customer-support traffic.

Conclusion. The experiment demonstrates that a small open-source LLM can be adapted efficiently to a narrow gaming-support intent taxonomy using LoRA, with a large improvement over the zero-shot baseline while training only 0.1093% of model parameters. 4-bit inference meaningfully reduced peak GPU memory, but did not guarantee lower latency and should not be assumed to improve accuracy in general.

Repository: [gaming-support-llm-finetuning](#) | Main notebook: [notebooks/gaming_intent_lora.ipynb](#)